BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 014 · Noughts & Crosses · Part 2 — Discover iteration

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. In a zero-indexed C# array, which element does array[2] access?

Q2. Name the three parts of a C# for loop header used to visit every array element.

Q3. In a C# 2D array organised as rows and columns, what does GetLength(0) return?

Q4. If a board has 3 rows and 3 columns, how many values does it contain?

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — 2D array

Recall the term — row

Recall the term — column

Recall the term — index

Recall the term — dimension

Recall the term — nested loop

Recall the term — GetLength()

New learning

Notice the repeated statements — Look at the three explicit row-printing instructions. The operation is the same; only the row index changes: 0, 1, 2 . — CODE:

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

Replace the rows with one loop — The first loop repeats the same row-print statement. row changes automatically; each repetition chooses a different row. — CODE:

```
for (int row = 0; row < board.GetLength(0); row++)
{
Console.WriteLine(board[row,0] + " | " +
board[row,1] + " | " +
board[row,2]);
}
```

Spot the second repetition — Inside each row, we still refer to columns 0, 1 and 2. A second loop can move across the columns of one row. — CODE:

```
for (int col = 0; col < board.GetLength(1); col++)
{
Console.Write(board[0,col]);
if (col < board.GetLength(1)-1)
Console.Write(" | ");
}
Console.WriteLine();
```

Put one loop inside the other — Outer loop = down the rows. Inner loop = across the columns. This is called nested iteration . — CODE:

```
for (int row = 0; row < board.GetLength(0); row++)
{
for (int col = 0; col < board.GetLength(1); col++)
{
Console.Write(board[row,col]);
if (col < board.GetLength(1)-1)
Console.Write(" | ");
}
Console.WriteLine();
}
```

Make it reusable — GetLength(0) gives the row count; GetLength(1) gives the column count. Place the loops in DrawBoard() , then call it whenever the display needs refreshing. — CODE:

```
DrawBoard(); // called from Main once the board exists
```

Worked example

From three statements to one loop, then two

1. Start with the old solution — CODE:

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

— It works. But it repeats the same pattern three times.

2. Replace the three row statements — CODE:

```
for (int row = 0; row < board.GetLength(0); row++)
{
Console.WriteLine(board[row,0] + " | " +
board[row,1] + " | " +
board[row,2]);
}
```

— Keep the row output the same but allow row to change inside the loop.

3. Replace the repeated columns — CODE:

```
for (int col = 0; col < board.GetLength(1); col++)
{
Console.Write(board[0,col]);
if (col < board.GetLength(1)-1)
Console.Write(" | ");
}
Console.WriteLine();
```

— Now each column is selected by col . After the third column, finish the row.

4. Combine both loops — CODE:
for (int row = 0; row < board.GetLength(0); row++)
{
for (int col = 0; col < board.GetLength(1); col++)
{
Console.Write(board[row,col]);
if (col < board.GetLength(1)-1)
Console.Write(" | ");
}
Console.WriteLine();
} — Predict how many times Console.Write(board[row,col]) executes: 9 .

5. Refine into DrawBoard() — CODE:
public static void DrawBoard()
{
for (int row = 0; row < board.GetLength(0); row++)
{
for (int col = 0; col < board.GetLength(1); col++)
{
Console.Write(board[row,col]);
if (col < board.GetLength(1)-1)
Console.Write(" | ");
}
Console.WriteLine();
}
} — This method assumes board is stored at class level. Call DrawBoard() from Main() . You can reuse it after every move in Lesson 015.

We Do · Practise with guidance

Identify the pattern

The original problem stays visible as we improve the code.

Write or predict

Study the original three row-print statements. Which part remains identical and which part changes?

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

What did we discover?

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

The code prints three values separated by pipes on every row. Only the row index changes (0 → 1 → 2).

Introduce a row loop

Complete the counter condition to visit rows 0, 1 and 2 in sequence.

```
for (int row = 0; row < ?; row++)
```

```
for (int row = 0; row < board.GetLength(0); row++)
```

Using the row count is better than assuming every board has three rows.

Notice the columns

For one row, write a loop that visits every column. Which dimension returns the number of columns?

```
for (int col = 0; col < ?; col++)
```

```
for (int col = 0; col < board.GetLength(1); col++)
```

The inner loop visits the columns across a single row.

Combine and explain

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Start with what worked
Use your three row-print statements from Lesson 014. Run them unchanged.

Question 2

Replace repeated rows
Write one for loop that prints all three rows. Within the loop you may still access columns 0, 1 and 2 explicitly.

Question 3

Replace repeated columns
Write a for loop to display one row, visiting columns using col . Print separators only between squares.

Question 4

Build DrawBoard()
Combine your two loops to visit all nine squares and put them inside DrawBoard() . Keep board accessible from the method and call it in Main() .

Exit Ticket · Answer before checking

board.GetLength(1) returns the number of columns in the board.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. The item at index 2 — the third item.

Q2. A counter, condition and update.

Q3. The number of rows.

Q4. 9.

You Do pages · Independent questions

Q1. 1. Re-run the sequential board
Begin with yesterday's three separate row statements before making any changes.
Confirm the expected output 1–9.
CODE:
```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

Q2. 2. Loop over the rows
Only the row index changes. Replace the three row statements with a loop:
CODE:
```
for (int row = 0; row < board.GetLength(0); row++)
{
Console.WriteLine(board[row,0] + " | " +
board[row,1] + " | " +
board[row,2]);
}
```
Check that output remains identical.

Q3. 3. Loop over the columns
Inside a row, the three column values are another repeated operation.
CODE:
```
for (int col = 0; col < board.GetLength(1); col++)
{
Console.Write(board[0,col]);
if (col < board.GetLength(1)-1)
```

```
Console.Write(" | ");
}
Console.WriteLine();
```

The row index is still 0 while col changes 0,1,2.

Q4. 4. Full DrawBoard method and runnable program

Place the board at class level and call DrawBoard() from Main() .

CODE:

```
using System;
public class Program
{
static string[,] board = {
{ "1", "2", "3" },
{ "4", "5", "6" },
{ "7", "8", "9" }
};

public static void DrawBoard()
{
for (int row = 0; row < board.GetLength(0); row++)
{
for (int col = 0; col < board.GetLength(1); col++)
{
Console.Write(board[row,col]);
if (col < board.GetLength(1)-1)
Console.Write(" | ");
}
Console.WriteLine();
}
}

public static void Main()
{
DrawBoard();
}
}
```

Expected: three rows; nine squares printed once. The method can be called after every move in Lesson 015.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Exit Ticket · Model answer

True. For a rectangular C# array, GetLength(1) gives its size in the second dimension: columns.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: